

Integration of Vision-Based Object Detection and Palletizing with the Dobot Magician Lite

Jatin Satyam¹

School for Engineering of Matter, Transport and Energy, Ira Fulton Schools of Engineering
Arizona State University, Tempe, Az

Article Info

Date of practical performed:

10/03/2025

Keywords:

YOLO
Food vs Vehicle
Pick-and-place

ABSTRACT

This paper presents the integration of computer vision and robotic manipulation for autonomous palletizing using the Dobot Magician Lite robot. A YOLO-based object detection system was implemented to classify objects into predefined categories (food vs. vehicle), and the robot executed suction-based pick-and-place operations to palletize them. The experiment demonstrated reliable perception-driven manipulation, highlighting the effectiveness of lightweight object detection frameworks for real-time robotic applications.

Corresponding Author:

Jatin Satyam
School for Engineering of Matter, Transport and Energy, Ira Fulton Schools of Engineering
Arizona State University, Tempe, Az
Email: jsatyam@asu.edu

1. INTRODUCTION

Robotic arms are essential in modern automation, enabling accurate and repeatable actions in industries such as packaging, logistics, and material handling. The Dobot Magician Lite provides an accessible, safe platform to explore these robotic functions through suction-based gripping and Python-controlled routines.

The innovation in this lab lies in combining YOLO-driven object detection with Dobot manipulation, allowing real-time classification to guide palletizing tasks. Students engaged with motion control, vision integration, and code-driven routines to achieve closed-loop autonomy.

Challenges such as serial communication delays, suction reliability, and precise workspace calibration revealed the trade-offs between perception and actuation, while also reinforcing skills in programming, system debugging, and control design. Overall, the lab highlights both the technical and instructional value of integrating modern vision algorithms with the Dobot Magician Lite in robotics and automation.

❖ Problem Statement:

The problem addressed in this lab is how real-time object detection can be combined with robotic manipulation to achieve autonomous palletizing. The assignment required using YOLO to classify objects as food or vehicle, commanding the Dobot Magician Lite to pick each item with a suction cup, and placing it into the correct pallet. Performance was measured in terms of classification accuracy, cycle consistency, and the reliability of sorting under repeated trials.

2. METHODOLOGY

The experimental platform was based on the Dobot Magician Lite, a compact robotic manipulator equipped with a suction gripper. The study simulated an automated sorting process under controlled laboratory conditions, integrating perception-driven object classification with palletizing. The end-effector was mounted with an external camera to capture images of objects placed in the workspace for real-time detection.

The robot was operated via an Ubuntu 22.04 LTS workstation to ensure compatibility, with communication established through Python 3.10 and the pydobot2 library. This framework provided high-level motion primitives (e.g., move_to) and effector commands (.suck()). Four robot poses were defined: a home pose, a pickup position, and two pallet drop zones corresponding to food and vehicle categories.

Plastic cubes labeled with representative object classes served as the workpieces, with each unit first detected and classified using YOLO before transfer. The robot then executed a suction-based pick-and-place

cycle, sequentially palletizing items into their designated bins. This setup enabled a systematic evaluation of perception-guided palletizing in terms of detection accuracy, cycle time, and placement consistency

3. RESULTS

The system successfully combined YOLO-based object detection with robotic palletizing on the Dobot Magician Lite. Across multiple trials, objects were consistently detected, classified, and placed into designated pallets with minimal error. Average cycle times were stable, placement accuracy remained within a few millimeters of the target, and overall reliability was high. Minor limitations were observed in suction grip consistency and detection confidence under varying lighting, but the system demonstrated robust end-to-end performance in vision-guided automation.

4. CONCLUSION

This lab demonstrated the successful integration of real-time object detection with robotic manipulation using the Dobot Magician Lite. By combining YOLO-based vision with suction-based pick-and-place operations, the system achieved reliable autonomous palletizing with consistent accuracy and repeatability. Minor challenges in lighting conditions and suction reliability highlighted practical constraints, but the overall results confirmed the effectiveness of perception-driven control in small-scale automation.

ACKNOWLEDGMENT

The author thanks Prof. Sangram Redkar for providing guidance and insights throughout the course, and the teaching assistant Sai Srinivas Tatwik Meesala and Rajesh S. Aouti for their assistance during Lab 3.

REFERENCES

[1] RAS 545 Laboratory Manual – Lab 3: Dobot Magician Lite Object Detection + Palletization.

- **APPENDIX**

Youtube video: <https://youtu.be/TPjBcd7Wd1E?si=4ma8CZJ4fvv5iaBz>

- PythonScripts:
camera_palletization.py

```

• import cv2
• import time
• from ultralytics import YOLO
• from pydobot.dobot import MODE_PTP
• import pydobot
•
• # ===== Robot Setup =====
• device = pydobot.Dobot(port="/dev/ttyACM0")
• device.speed(10, 10)
• device.home()
•
• # ===== Camera Setup =====
• cap = cv2.VideoCapture(0)
• if not cap.isOpened():
•     raise RuntimeError("Cannot open camera")
• cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
• cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

```

```

•
• # ===== YOLO Setup =====
• model = YOLO("yolov8s.pt")
•
• # ===== Robot Coordinates (adjust!) =====
• home_coordinates = [239.999, 0.0, 150.0, -8.881]
• intermediate_coordinates = [193.526, 22.005, 35.189, 6.487]
• pickup_coordinates = [306.420, -0.706, -55.166, -15.104]
• palletteA_coordinates = [210.180, -233.553, 24.075, -46.766]
• palletteB_coordinates = [296.191, 193.261, 25.764, 33.123]
•
• def is_home():
•     """Check if robot is at home position"""
•     pose = device.get_pose()
•     try:
•         current_pose = (pose.position.x, pose.position.y,
• pose.position.z, pose.position.r)
•     except AttributeError:
•         try:
•             current_pose = (pose.x, pose.y, pose.z, pose.r)
•         except AttributeError:
•             current_pose = tuple(pose[:4]) if hasattr(pose, '__iter__')
• else pose
•
•     home_pose = tuple(home_coordinates)
•     position_tol = 10 # mm tolerance
•     is_at_home = all(abs(current_pose[i] - home_pose[i]) < position_tol
• for i in range(3))
•     return is_at_home
•
• # ===== Main Loop =====
• try:
•     print("Starting object detection and palletization...")
•     print("Press 'q' to quit")
•
•     cv2.namedWindow('Camera Feed', cv2.WINDOW_NORMAL)
•
•     detection_enabled = True # Flag to pause detection during robot
moves
•
•     while True:
•         ret, frame = cap.read()
•         if not ret:
•             print("Camera error, skipping frame...")
•             continue
•
•         annotated_frame = frame.copy()
•         detected_class, detected_label = None, "None"

```

```

•
• # ===== Run YOLO Only When Enabled =====
• if detection_enabled and is_home():
•     results = model(frame, verbose=False)
•     r = results[0]
•     annotated_frame = r.plot()
•
•     if r.bboxes is not None and len(r.bboxes) > 0:
•         names = r.names
•         for box in r.bboxes:
•             cls_id = int(box.cls[0])
•             conf = float(box.conf[0])
•             label = names[cls_id].lower()
•
•             print(f"Detected: {label} (confidence: {conf:.2f})")
•
•             x1, y1, x2, y2 = map(int, box.xyxy[0])
•             cv2.putText(annotated_frame,
•                         f"{label} {conf:.2f}",
•                         (x1, y1 - 10),
•                         cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255,
0), 2)
•
•             if conf > 0.75:
•                 if label in ["car", "bicycle", "airplane"]:
•                     detected_class = "vehicle"
•                     detected_label = f"{label} ({conf:.2f})"
•                     break
•                 elif label in ["apple", "banana", "pizza"]:
•                     detected_class = "food"
•                     detected_label = f"{label} ({conf:.2f})"
•                     break
•
• # Display status on feed
• status_text = f"Status: {detected_class if detected_class else
'Waiting...'}"
• label_text = f"Object: {detected_label}"
• cv2.putText(annotated_frame, status_text, (10, 30),
•             cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
• cv2.putText(annotated_frame, label_text, (10, 70),
•             cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
•
• cv2.imshow('Camera Feed', annotated_frame)
•
• # Quit key
• if cv2.waitKey(1) & 0xFF == ord('q'):
•     print("Quit key pressed")

```

```

    break

    # ===== Robot Actions =====
    if detected_class == "food":
        print("Food detected → moving to Pallet A")
        detection_enabled = False

        device.move_to(mode=int(MODE_PTP.MOVJ_XYZ),
            x=pickup_coordinates[0],
            y=pickup_coordinates[1],
            z=pickup_coordinates[2], r=pickup_coordinates[3])
        time.sleep(0.3)
        device.suck(True)
        time.sleep(0.5)

        device.move_to(mode=int(MODE_PTP.MOVJ_XYZ),
            x=intermediate_coordinates[0],
            y=intermediate_coordinates[1],
            z=intermediate_coordinates[2], r=intermediate_coordinates[3])

        device.move_to(mode=int(MODE_PTP.MOVJ_XYZ),
            x=palletteA_coordinates[0],
            y=palletteA_coordinates[1],
            z=palletteA_coordinates[2], r=palletteA_coordinates[3])
        time.sleep(0.3)
        device.suck(False)
        time.sleep(0.5)

        device.home()
        time.sleep(2)

        detection_enabled = True

    elif detected_class == "vehicle":
        print("Vehicle detected → moving to Pallet B")
        detection_enabled = False

        device.move_to(mode=int(MODE_PTP.MOVJ_XYZ),
            x=pickup_coordinates[0],
            y=pickup_coordinates[1],
            z=pickup_coordinates[2], r=pickup_coordinates[3])
        time.sleep(0.3)
        device.suck(True)
        time.sleep(0.5)

        device.move_to(mode=int(MODE_PTP.MOVJ_XYZ),
            x=intermediate_coordinates[0],

```

```
•         y=intermediate_coordinates[1],
•         z=intermediate_coordinates[2], r=intermediate_coordinates[3])
•
•         device.move_to(mode=int(MODE_PTP.MOVJ_XYZ),
•         x=palletteB_coordinates[0],
•         y=palletteB_coordinates[1],
•         z=palletteB_coordinates[2], r=palletteB_coordinates[3])
•         time.sleep(0.3)
•         device.suck(False)
•         time.sleep(0.5)
•
•         device.home()
•         time.sleep(2)
•
•         detection_enabled = True
•
•     else:
•         time.sleep(0.05)
•
• except KeyboardInterrupt:
•     print("\nStopping program...")
• except Exception as e:
•     print(f"Error occurred: {e}")
•     import traceback
•     traceback.print_exc()
• finally:
•     cv2.destroyAllWindows()
•     if cap is not None:
•         cap.release()
•     device.close()
•     print("Cleanup complete")
```
